

1. What is String Constant Pool?

It is a special memory area used to store String literals. It allows Java to reuse String objects instead of creating duplicated objects. For example, if 2 variables are assigned the same String literal, they will reference the same objects in String constant pool.

String constant pool is stored in the heap.

String is immutable. Once it is created, its value cannot be changed & the object cannot be modified. Thus, Java can safely share the same String object from String Constant pool without worrying about that one reference will affect another.

2. Diff between equals() and ==

In Java, double equal sign compares memory address. It checks whether two references point to the same object in memory.

equals() in default also compares memory addresses. It is inherited from the Object class. But, classes like String or Integer, override it to compare the actual content instead.

For custom class, we need to override equals() ourselves if we want to compare contents.

In Java, hashCode() returns an integer used by Hash Map to determine where the bucket stores the object.

Hash Collision occurs when multiple keys end up in the same buckets, so hashCode() calls equals() to distinguish them and find the exact matching key.

This is why we need to override equals(). If we do not override equals when we compare actual contents, two same contents might return false using the comparison of equals().

3. How to Handle race condition?

Race Condition happens when multiple threads access & modify the same shared resource at the same time. This might lead to unpredictable results. We handle it in several ways.

1) Using the Synchronized keyword: It restricts the access to the shared resource, so that only one thread can get access to the resource at a time.

But the disadvantage is that the waiting threads have to be blocked infinitely. There is no way to timeout, interrupt the waiting.

2) Using Lock interface & ReentrantLock implementation. It restricts the access of multiple locks, allowing for only one thread access. But it allows for more control. We can use lock() for access restriction, but unlock() to manually release the blocking thread. Trylock() also allows for a timeout so that the waiting thread can give up if waiting for a long time.

3) Using Atomic classes such as AtomicInteger, AtomicLong to handle thread safety without and lock. It has a low-level instructions called compare-and-swap (CAS) to ensure the read-modify-write operation are performed atomically. It is quick, but only works on simple single variable operations.

4) The forth way is to use volatile keyword and Double-checked Locking Singleton pattern. we use synchronize keyword inside of the pattern. We also use double "null" to prevent duplicate creation.

4. Thread life cycle / Thread State.

A Thread in Java goes through 6 states.

① It starts in the NEW state when a thread object is created. But it will not run until start() is called.

② Once start() is called, the thread moves to Runnable state, meaning it is either running or ready to run and waiting for the CPU to schedule it.

③ A thread enters the Timed Waiting State if a thread calls sleep() or join() with a timeout.

A thread enters Waiting state if it calls wait() & join() without timeout.

A thread enters Blocked state if it enters a synchronized block which is already locked by another ^{State}.

④ When a thread finishes executing, it moves to Terminated state.

1. What is Optional Class and how to use it?

Optional is a wrapped class introduced in Java 8. It holds a value which may or may not be present. It handles to eliminate `NullPointerException`.

Instead of returning a null, we return a `Optional.of(value)` when a value exists or

`Optional.empty()` when it does not.

Common methods include `isPresent()` to check if a value exists, `orElse()` to provide a default value when empty. `orElseThrow()` throws an exception when empty.

`ifPresent()` executes logic only when a value is present.

Again. What is String Constant pool?

It is a special memory area used to store String Literals. It allows Java to re-use

Strings rather than creating duplicated one. If two strings have the same String Literals, they will point to the same object in String pool. String constant pool is

stored in the Heap.

String is immutable. Once it is created, its value won't be changed and the object cannot be modified.